

Digital Radio Lab

Experiment: Combinational Logic Circuits

Name	Harsh Kumar
Roll No.	226EC5003
Subject	Digital Radio Lab
Experiment	Combinational Logic Circuits
Tool Used	EDA Playground (Verilog HDL)

1. Aim

- To implement and verify all the basic logic gates: BUFFER, AND, OR, NOT, NAND, NOR, XOR and XNOR.
- To design and implement a 4x1 Multiplexer using 2x1 Multiplexers, and a 1x4 Demultiplexer using 1x2 Demultiplexers.
- To design and implement an 8x3 Encoder and a 3x8 Decoder.

2. Software Used

- EDA Playground (online Verilog/SystemVerilog simulator)
- Simulator: Icarus Verilog 12.0 (or any simulator available on EDA Playground)
- Waveform viewer: EPWave
- Language: Verilog HDL

3. Theory

Combinational circuits are digital circuits whose output depends only on the present values of the inputs and not on past inputs (no memory).

3.1 Basic Logic Gates

Logic gates are the basic building blocks of digital systems. A BUFFER passes the input unchanged ($Y = A$). A NOT gate inverts the input ($Y = A'$). AND gives 1 only when all inputs are 1 ($Y = A.B$). OR gives 1 when at least one input is 1 ($Y = A+B$). NAND and NOR are the complements of AND and OR respectively. XOR gives 1 when inputs differ ($Y = A \oplus B$) and XNOR gives 1 when inputs are the same ($Y = (A \oplus B)'$).

3.2 Multiplexer (MUX)

A multiplexer selects one of several data inputs and forwards it to a single output, based on select lines. A 2x1 MUX has the equation $Y = S'I_0 + S.I_1$. A 4x1 MUX needs two select lines (S_1, S_0) and can be built from three 2x1 MUXes: two in the first stage (selected by S_0) and one in the second stage (selected by S_1). $Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$.

3.3 Demultiplexer (DEMUX)

A demultiplexer takes a single input and routes it to one of several outputs according to the select lines. For a 1x2 DEMUX: $Y_0 = D.S'$ and $Y_1 = D.S$. A 1x4 DEMUX is built using three 1x2 DEMUXes: the first stage (select S_1) splits D into two lines, and the second stage (select S_0) splits each line again.

3.4 Encoder (8x3)

An encoder converts 2^n input lines into an n -bit binary code. In an 8x3 encoder only one input is active (high) at a time, and the output is the binary code of that input: $A_2 = D_4 + D_5 + D_6 + D_7$, $A_1 = D_2 + D_3 + D_6 + D_7$, $A_0 = D_1 + D_3 + D_5 + D_7$.

3.5 Decoder (3x8)

A decoder converts an n -bit binary input into 2^n unique output lines, of which exactly one is active for each input combination. Each output is a minterm of the inputs, e.g. $Y_0 = A_2'A_1'A_0'$, $Y_7 = A_2A_1A_0$.

Note: The question mentions a '3x8 encoder and 8x3 decoder'; the standard definitions are an 8x3 encoder (8 inputs, 3 outputs) and a 3x8 decoder (3 inputs, 8 outputs), which are implemented here.

4. Source Code (Verilog)

In EDA Playground, the code is written in two separate windows: the **design** module goes in **design.sv** and the **testbench** goes in **testbench.sv**. Each experiment below is therefore given as two separate listings.

4.1 Basic Logic Gates

design.sv (Design window - right side in EDA Playground)

```
1 // ===== DESIGN: Basic Logic Gates =====
2 module gates (
3     input  a, b,
4     output y_buf, y_not, y_and, y_or,
5     output y_nand, y_nor, y_xor, y_xnor
6 );
7     assign y_buf  = a;
8     assign y_not  = ~a;
9     assign y_and  = a & b;
10    assign y_or   = a | b;
11    assign y_nand = ~(a & b);
12    assign y_nor  = ~(a | b);
13    assign y_xor  = a ^ b;
14    assign y_xnor = ~(a ^ b);
15 endmodule
```

testbench.sv (Testbench window - left side in EDA Playground)

```
1 // ===== TESTBENCH =====
2 module gates_tb;
3     reg a, b;
4     wire y_buf, y_not, y_and, y_or, y_nand, y_nor, y_xor, y_xnor;
5
6     gates dut (a, b, y_buf, y_not, y_and, y_or,
7                y_nand, y_nor, y_xor, y_xnor);
8
9     integer k;
10
11    initial begin
12        $dumpfile("dump.vcd");
13        $dumpvars(0, gates_tb);
14
15        $display("A B | BUF NOT AND OR NAND NOR XOR XNOR");
16        for (k = 0; k < 4; k = k + 1) begin
17            {a, b} = k;
18            #10;
19            $display("%b %b | %b %b %b %b %b %b %b %b %b",
20                    a, b, y_buf, y_not, y_and, y_or,
21                    y_nand, y_nor, y_xor, y_xnor);
22        end
23        $finish;
24    end
25 endmodule
```

4.2 4x1 Multiplexer using 2x1 Multiplexers

design.sv (Design window - right side in EDA Playground)

```
1 // ===== DESIGN: 2x1 MUX =====
2 module mux2x1 (
3     input i0, i1, s,
4     output y
5 );
6     assign y = s ? i1 : i0;
7 endmodule
```

```

8
9 // ===== DESIGN: 4x1 MUX using 2x1 MUX =====
10 module mux4x1 (
11     input i0, i1, i2, i3,
12     input s1, s0,
13     output y
14 );
15     wire w0, w1;
16
17     mux2x1 m0 (.i0(i0), .i1(i1), .s(s0), .y(w0));
18     mux2x1 m1 (.i0(i2), .i1(i3), .s(s0), .y(w1));
19     mux2x1 m2 (.i0(w0), .i1(w1), .s(s1), .y(y));
20 endmodule

```

testbench.sv (Testbench window - left side in EDA Playground)

```

1 // ===== TESTBENCH =====
2 module mux4x1_tb;
3     reg i0, i1, i2, i3, s1, s0;
4     wire y;
5
6     mux4x1 dut (i0, i1, i2, i3, s1, s0, y);
7
8     integer k;
9
10    initial begin
11        $dumpfile("dump.vcd");
12        $dumpvars(0, mux4x1_tb);
13
14        // fixed data inputs
15        i0 = 1; i1 = 0; i2 = 1; i3 = 1;
16
17        $display("S1 S0 | Y");
18        for (k = 0; k < 4; k = k + 1) begin
19            {s1, s0} = k;
20            #10;
21            $display("%b %b | %b", s1, s0, y);
22        end
23        $finish;
24    end
25 endmodule

```

4.3 1x4 Demultiplexer using 1x2 Demultiplexers

design.sv (Design window - right side in EDA Playground)

```

1 // ===== DESIGN: 1x2 DEMUX =====
2 module demux1x2 (
3     input d, s,
4     output y0, y1
5 );
6     assign y0 = d & ~s;
7     assign y1 = d & s;
8 endmodule
9
10 // ===== DESIGN: 1x4 DEMUX using 1x2 DEMUX =====
11 module demux1x4 (
12     input d, s1, s0,
13     output y0, y1, y2, y3
14 );
15     wire w0, w1;
16
17     demux1x2 d0 (.d(d), .s(s1), .y0(w0), .y1(w1));
18     demux1x2 d1 (.d(w0), .s(s0), .y0(y0), .y1(y1));
19     demux1x2 d2 (.d(w1), .s(s0), .y0(y2), .y1(y3));
20 endmodule

```

testbench.sv (Testbench window - left side in EDA Playground)

```
1 // ===== TESTBENCH =====
2 module demux1x4_tb;
3   reg d, s1, s0;
4   wire y0, y1, y2, y3;
5
6   demux1x4 dut (d, s1, s0, y0, y1, y2, y3);
7
8   integer k;
9
10  initial begin
11    $dumpfile("dump.vcd");
12    $dumpvars(0, demux1x4_tb);
13
14    d = 1;
15    $display("D S1 S0 | Y0 Y1 Y2 Y3");
16    for (k = 0; k < 4; k = k + 1) begin
17      {s1, s0} = k;
18      #10;
19      $display("%b %b %b | %b %b %b %b",
20              d, s1, s0, y0, y1, y2, y3);
21    end
22    $finish;
23  end
24 endmodule
```

4.4 8x3 Encoder

design.sv (Design window - right side in EDA Playground)

```
1 // ===== DESIGN: 8x3 Encoder =====
2 module encoder8x3 (
3   input [7:0] d,
4   output reg [2:0] y
5 );
6   always @(*) begin
7     case (d)
8       8'b00000001 : y = 3'b000;
9       8'b00000010 : y = 3'b001;
10      8'b00000100 : y = 3'b010;
11      8'b00001000 : y = 3'b011;
12      8'b00010000 : y = 3'b100;
13      8'b00100000 : y = 3'b101;
14      8'b01000000 : y = 3'b110;
15      8'b10000000 : y = 3'b111;
16      default      : y = 3'bxxx; // invalid input
17    endcase
18  end
19 endmodule
```

testbench.sv (Testbench window - left side in EDA Playground)

```
1 // ===== TESTBENCH =====
2 module encoder8x3_tb;
3   reg [7:0] d;
4   wire [2:0] y;
5
6   encoder8x3 dut (d, y);
7
8   integer k;
9
10  initial begin
11    $dumpfile("dump.vcd");
12    $dumpvars(0, encoder8x3_tb);
13
14    $display("D7..D0 | Y2 Y1 Y0");
```

```

15     for (k = 0; k < 8; k = k + 1) begin
16         d = 8'b1 << k;
17         #10;
18         $display("%b | %b %b %b", d, y[2], y[1], y[0]);
19     end
20     $finish;
21 end
22 endmodule

```

4.5 3x8 Decoder

design.sv (Design window - right side in EDA Playground)

```

1  // ===== DESIGN: 3x8 Decoder =====
2  module decoder3x8 (
3      input      [2:0] a,
4      input      en,
5      output reg [7:0] y
6  );
7      always @(*) begin
8          y = 8'b00000000;
9          if (en)
10             y[a] = 1'b1;
11      end
12 endmodule

```

testbench.sv (Testbench window - left side in EDA Playground)

```

1  // ===== TESTBENCH =====
2  module decoder3x8_tb;
3      reg [2:0] a;
4      reg      en;
5      wire [7:0] y;
6
7      decoder3x8 dut (a, en, y);
8
9      integer k;
10
11     initial begin
12         $dumpfile("dump.vcd");
13         $dumpvars(0, decoder3x8_tb);
14
15         en = 1;
16         $display("A2 A1 A0 | Y7..Y0");
17         for (k = 0; k < 8; k = k + 1) begin
18             a = k;
19             #10;
20             $display("%b %b %b | %b", a[2], a[1], a[0], y);
21         end
22
23         // enable low -> all outputs 0
24         en = 0; a = 3'b101;
25         #10 $display("EN=0 -> Y = %b", y);
26         $finish;
27     end
28 endmodule

```

5. Procedure

1. Open EDA Playground (edaplayground.com) and log in.
2. Select **Testbench + Design** language as **SystemVerilog/Verilog** and choose the simulator (Icarus Verilog).
3. Type the design module(s) in the **design.sv** window and the corresponding testbench in the **testbench.sv** window (the testbench is not to be mixed with the design code).
4. Tick **Open EPWave after run** and click **Run**.
5. Observe the truth table printed in the log/console window and verify it against the theoretical truth table.
6. Open EPWave and observe the output waveforms for each input combination.
7. Repeat steps 3-6 for the basic gates, 4x1 MUX, 1x4 DEMUX, 8x3 encoder and 3x8 decoder.
8. Take screenshots of the console output and waveforms and attach them to the report.

6. Observations

6.1 Basic Logic Gates

A	B	BUFFER	NOT(A)	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	1	0	0	1	1	0	1
0	1	0	1	0	1	1	0	1	0
1	0	1	0	0	1	1	0	1	0
1	1	1	0	1	1	0	0	0	1

6.2 4x1 Multiplexer (data inputs fixed: I0=1, I1=0, I2=1, I3=1)

S1	S0	Selected input	Output Y
0	0	I0	1
0	1	I1	0
1	0	I2	1
1	1	I3	1

6.3 1x4 Demultiplexer (D = 1)

D	S1	S0	Y0	Y1	Y2	Y3
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

6.4 8x3 Encoder

D7	D6	D5	D4	D3	D2	D1	D0	Y2	Y1	Y0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1

D7	D6	D5	D4	D3	D2	D1	D0	Y2	Y1	Y0
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

6.5 3x8 Decoder (EN = 1)

A2	A1	A0	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

6.6 Simulation Output (Screenshots)

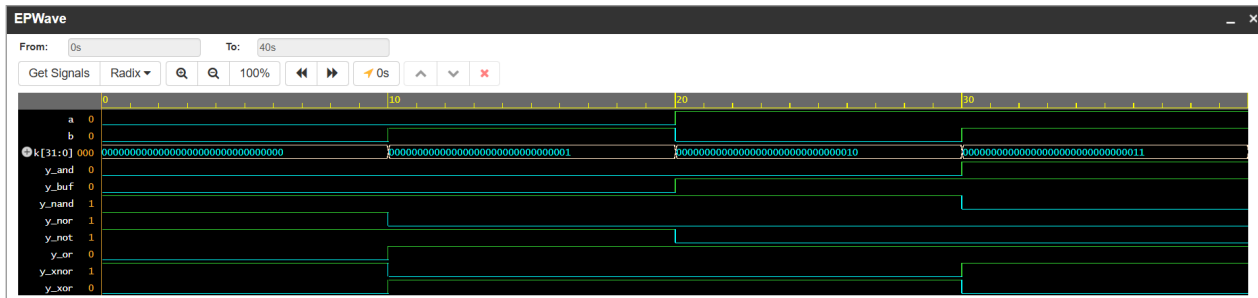


Fig. 6.1: Basic logic gates - EPWave output (A, B and all gate outputs)

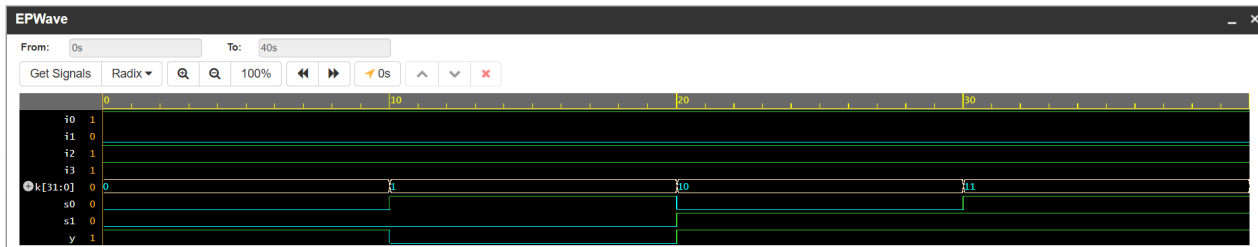


Fig. 6.2: 4x1 Multiplexer using 2x1 Multiplexers - EPWave output

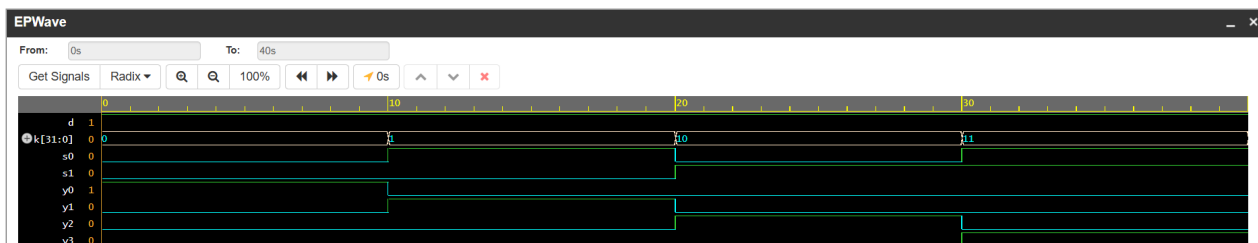


Fig. 6.3: 1x4 Demultiplexer using 1x2 Demultiplexers - EPWave output

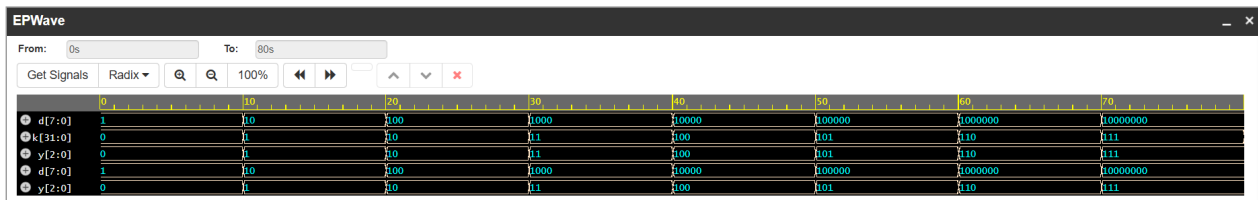


Fig. 6.4: 8x3 Encoder - EPWave output

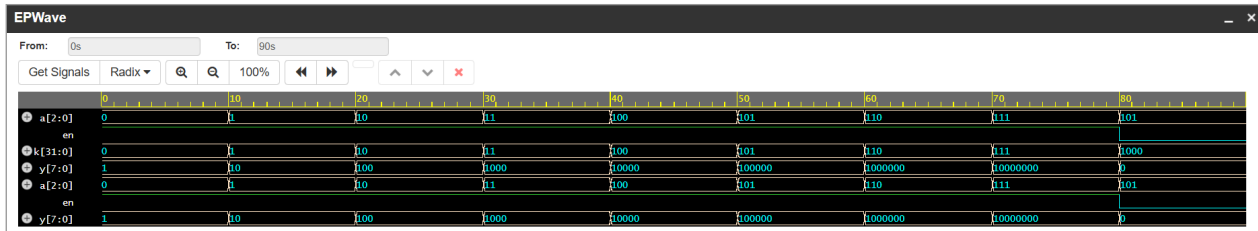


Fig. 6.5: 3x8 Decoder - EPWave output (including EN = 0 case)

7. Results & Discussion

- The outputs of all basic gates (BUFFER, NOT, AND, OR, NAND, NOR, XOR, XNOR) obtained from simulation matched their theoretical truth tables.
- The 4x1 MUX built from three 2x1 MUXes correctly routed the selected data input (I0-I3) to the output for every combination of S1 and S0.
- The 1x4 DEMUX built from three 1x2 DEMUXes routed the input D to exactly one output line chosen by S1 and S0, while all other outputs remained 0.
- The 8x3 encoder produced the correct 3-bit binary code for each active input line. Inputs with more than one active line are treated as invalid (a priority encoder is needed to handle them).
- The 3x8 decoder activated exactly one of the eight outputs for each 3-bit input, and all outputs were 0 when the enable input was low.
- Since all circuits are combinational, outputs changed immediately (after the simulated delay) whenever the inputs changed, with no dependence on previous states.

8. Conclusion

All the basic logic gates were implemented in Verilog and verified using EDA Playground. A 4x1 multiplexer and a 1x4 demultiplexer were successfully designed using smaller 2x1 and 1x2 blocks respectively, showing the concept of hierarchical design. The 8x3 encoder and 3x8 decoder were also designed and their functionality was verified through simulation. The simulation results matched the theoretical truth tables, hence the experiment was successfully completed.